

LOST AND FOUND MANAGEMENT SYSTEM



In Fulfillment of the Requirements for
CC003.23 Computer Programming 2
Bachelor of Science in Computer Science

Submitted by:

Elizan, Ramon Benedict R.

Iranon, Johanna Ezra

Guadalquiver, Art Xenos D.

Section:

BSCS – 1A

Submitted to:

Ma'am Aurora Manseras

A. Introduction

The **Lost and Found Management System** is a Java Swing-based desktop application designed to efficiently manage and organize lost and found items within a school or institutional environment. The system provides a centralized and user-friendly platform where users can record, monitor, and update item information in a structured manner.

Unlike traditional paper-based tracking methods, which are prone to loss, duplication, and human error, this system utilizes a digital table interface to store records systematically. This allows faster data retrieval, improved organization, and more accurate monitoring of item status, particularly in determining whether an item has been found or successfully returned to its rightful owner.

Purpose of the Application

The primary purpose of this system is to streamline and simplify the process of handling lost and found items. It aims to improve efficiency and reliability by replacing manual processes with a digital solution. Specifically, the system is designed to:

- Digitally store and organize item records in a structured format
- Track the status of items, including whether they are found or retrieved
- Minimize errors commonly associated with manual record-keeping
- Provide a convenient way to save and load data using JSON files for future access

Target Users

This system is intended for use by individuals responsible for managing lost and found operations within an organization. These include:

- School administrators
- Lost-and-found personnel
- Student assistants or designated staff members

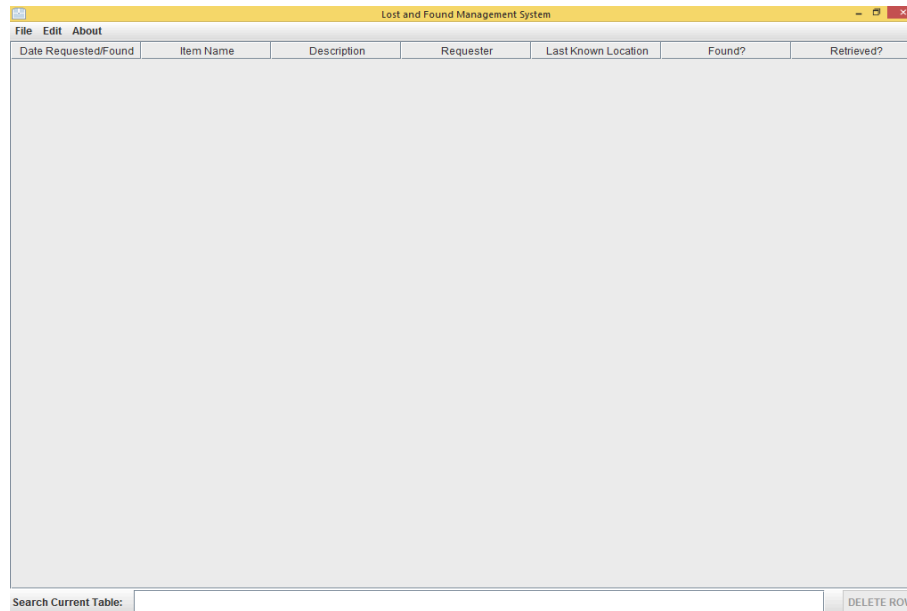
These users are tasked with encoding new records, updating item information, and maintaining accurate tracking of lost and found items to ensure efficient management and retrieval processes.

B. User Manual

This section provides a clear step-by-step guide on how to operate and navigate the system effectively.

1. Launching the System

To start the application, “Run” the Java program using an Integrated Development Environment (IDE) such as NetBeans. Once executed, the main window will appear, displaying the primary interface which includes the data table, menu bar, and toolbar. This serves as the central workspace where all operations are performed.

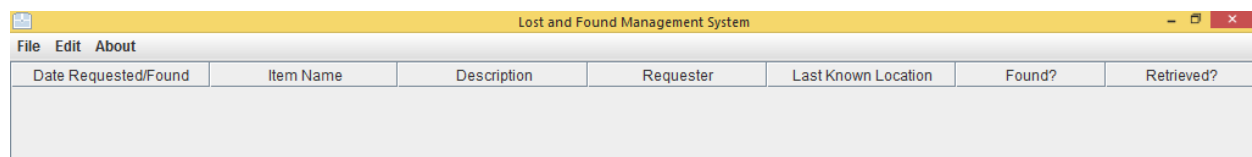


2. Main Interface (Table View)

The main interface presents all records in a structured table format. Each row represents a lost or found item, while the columns contain the following details:

- Date Requested/Found
- Item Name
- Description
- Requester
- Last Known Location
- Found? (checkbox)
- Retrieved? (checkbox)

Users can easily organize data by clicking on the column headers, which enables sorting in ascending or descending order. The table also supports scrolling and displays long text using automatic text wrapping.

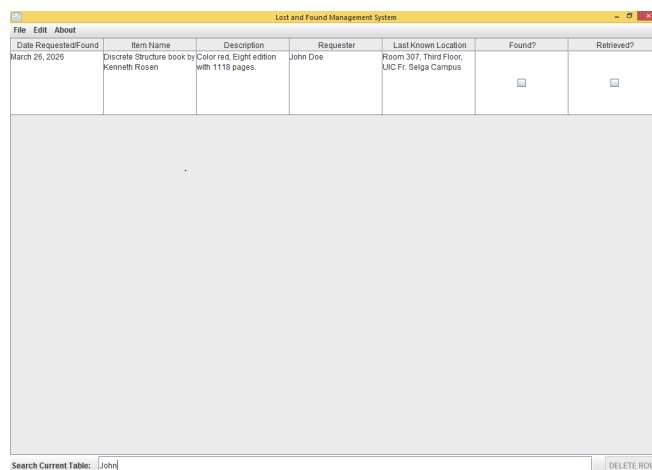
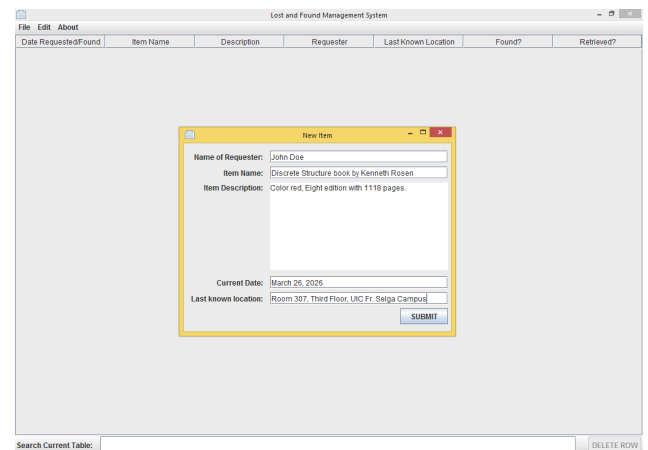
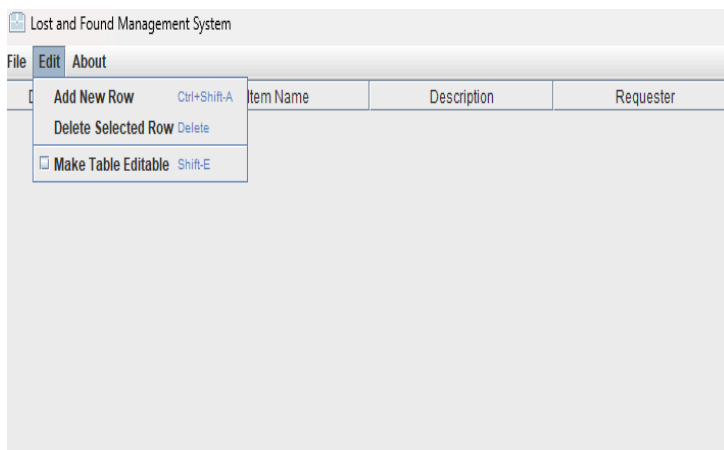


3. Adding a New Record

To add a new entry:

- Navigate to **Edit** → **Add New Item** or press **Ctrl + Shift + A**
- A form window will appear prompting the user to input item details
- Fill in all required fields:
 - Requester Name
 - Item Name
 - Description
 - Date
- The **Location** field is optional and will automatically be set to “N/A” if left blank
- Click **Submit** to save the record into the table

The system includes input validation and will disable the submit button if any required field is left empty.

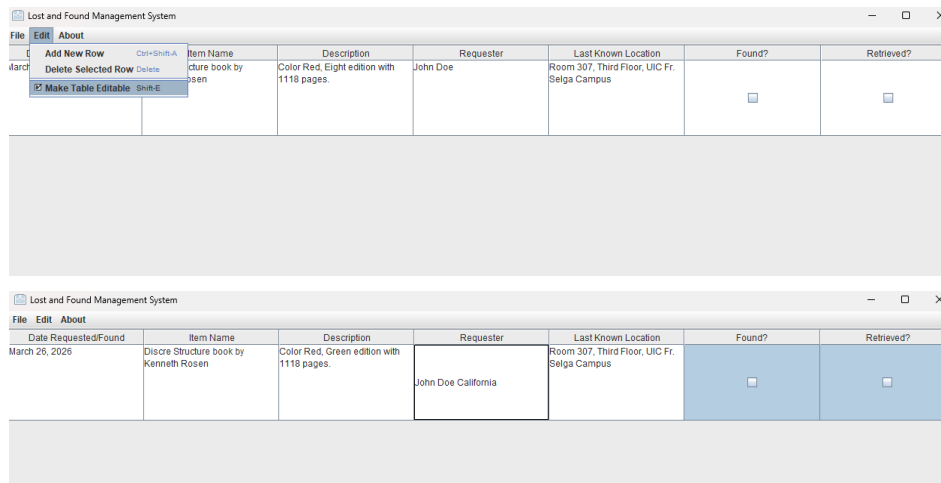


4. Editing Records

To modify existing records:

- Enable editing through **Edit** → **Make Table Editable (Shift + E)**
- When enabled, all table fields can be directly edited
- When disabled, only the “**Found?**” and “**Retrieved?**” columns remain editable for quick status updates

This feature ensures both flexibility and data protection.

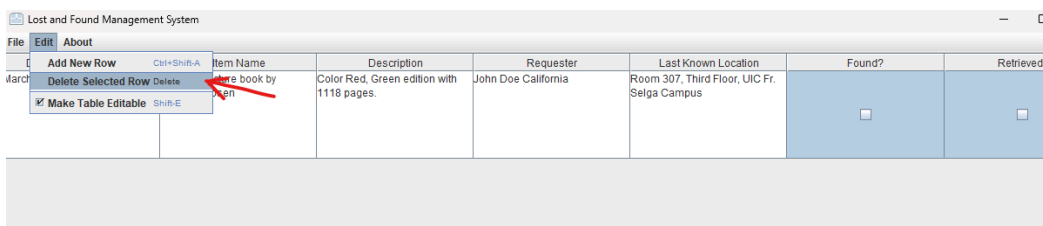


5. Deleting a Record

To remove a record:

- Select a row from the table
- Click the **Delete Row** button or press the **Delete** key
- The selected record will be permanently removed from the table

The delete button is only enabled when a row is selected to prevent accidental deletion.



6. Searching Records

The system provides a real-time search feature:

- Locate the search bar in the
- Enter any keyword (e.g., item name, requester, or description)

- The table will automatically filter matching records as you type

The search function is case-insensitive and updates instantly without requiring additional actions.

Date Requested/Found	Item Name	Description	Requester	Last Known Location	Found?	Retrieved?
March 26, 2026	Smartphone (Vivo)	Vivo Y29, color brown with old yellowish phone case	Art Xenos	UIC Fr. Seiga, Library	☐	☐

Search Current Table: DELETE ROW

Date Requested/Found	Item Name	Description	Requester	Last Known Location	Found?	Retrieved?
March 26, 2026	Discrete Structure book by Kenneth Rosen	Color red, Eight edition with 1118 pages	John Doe	Room 307, Third Floor, UIC Fr. Seiga Campus	☐	☐

Search Current Table: DELETE ROW

7. Saving Data

To save current records:

- Go to **File** → **Save to JSON (Ctrl + S)**
- Choose the desired file location
- The system will store all table data in a **.json** file

If the file name does not include “.json”, the system will automatically append it.

Date Requested/Found	Item Name	Description	Requester	Last Known Location	Found?	Retrieved?
March 26, 2026	Discrete Structure book by Kenneth Rosen	Color Red, Green edition with 1118 pages	John Doe California	Room 307, Third Floor, UIC Fr. Seiga Campus	☐	☐

File menu options: New Table, Save to JSON, Load from JSON, Exit.

Save in:

File Name:

Files of Type:

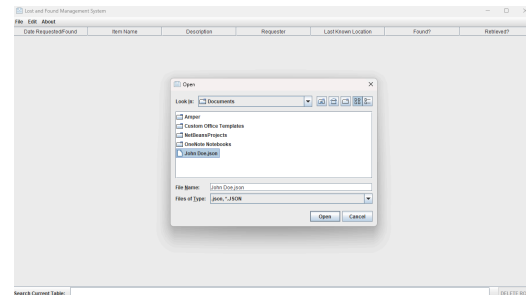
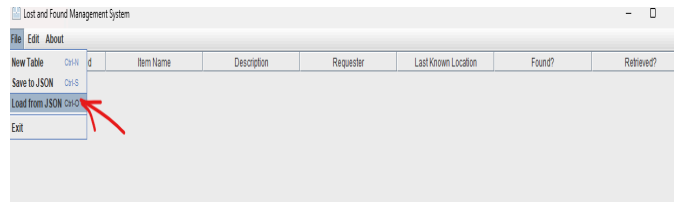
Save Cancel

8. Loading Data

To load previously saved records:

- Go to **File** → **Load from JSON (Ctrl + O)**
- Select a valid JSON file
- Confirm the action when prompted

The system will then populate the table with the loaded data. Any existing unsaved data will be overwritten.

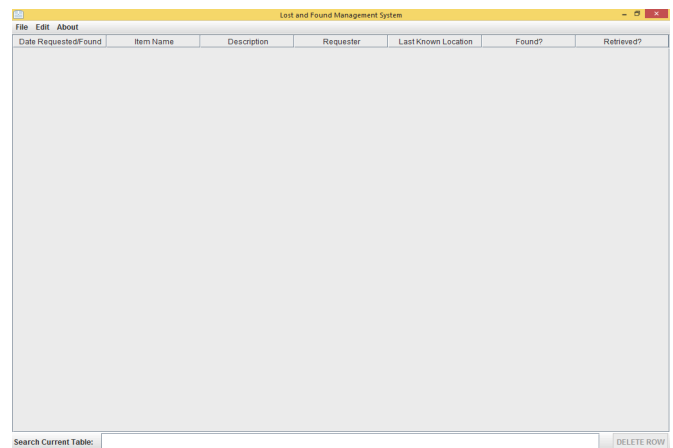
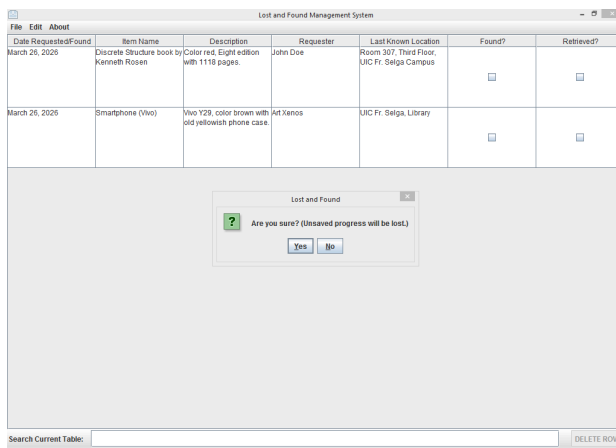


9. Creating a New Table

To clear all records:

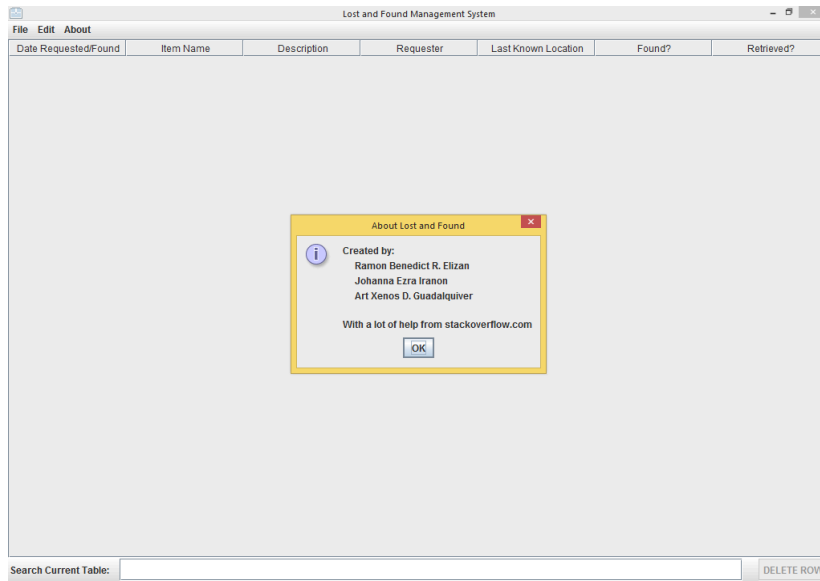
- Select **File** → **New Table (Ctrl + N)**
- Confirm the action

This will reset the table and remove all current entries.



10. About Section

The About section displays information about the developers and provides basic details about the system. It can be accessed through the menu bar.



C. Feature List

The system offers a range of functionalities designed to enhance usability and efficiency.

1. Core Functionalities (CRUD)

- **Create:** Add new lost and found records through a user-friendly form
- **Read:** View all records in an organized table
- **Update:** Edit item details and update status
- **Delete:** Remove selected entries from the system

2. Search and Filtering

- Real-time search functionality
- Case-insensitive filtering
- Instant updates through dynamic table sorting

3. Data Persistence (JSON System)

- Save records into JSON format
- Load previously saved data
- Automatic file extension handling

4. User Interface Features

- Scrollable table display (JTable)
- Text wrapping for long descriptions
- Sortable columns for better organization

- Toolbar with integrated search and delete options
- Keyboard shortcuts for faster navigation

5. Data Validation and Safety

- Prevents submission of incomplete forms
- Automatically assigns default values for optional fields
- Displays confirmation dialogs before:
 - Clearing data
 - Loading files
 - Exiting the application

6. Status Tracking System

- Uses checkboxes to track item progress:
 - Found
 - Retrieved
- Enables quick and clear monitoring of each item's status

D. Code Documentation

The application is developed using **Java Swing** and follows an **event-driven programming approach**, where user interactions trigger specific system behaviors. The program is implemented in a single main class and integrates GUI components, data handling, and event management.

1. Main Class

```
public class ElizanIranonGuadalquiver_FinalProject_CC003
```

This class serves as the core of the application and contains all components and logic of the system. It manages the graphical user interface, event handling, and data processing. The program primarily uses static variables and methods for centralized control and simplicity.

2. GUI Components

The system is built using several key Swing components:

- **JFrame (f)**
The main window of the application, which holds all interface elements.
- **JTable (MainTable) and DefaultTableModel (Table1)**
Used to display and manage all item records in a tabular format.
The table includes custom behavior such as:

- Restricting which columns can be edited
- Displaying “Found” and “Retrieved” as Boolean checkboxes

```
public static void Frame() {
    f.setSize(1280, 720);
    f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    f.setLocationRelativeTo(null); //jframe will appear in the center

    Toolkit k = Toolkit.getDefaultToolkit();
    Image Icon = k.createImage("l&f.png");
    f.setIconImage(Icon);
}
```

```
static JTable MainTable = new JTable(Table1); //apply tablemodel to MainTable
```

3. Table Model Customization

The table model is customized to improve usability and data control:

- **Editable Logic (isCellEditable)**
Controls which cells can be edited.
 - All cells become editable when the “Make Table Editable” option is enabled
 - Otherwise, only the “Found?” and “Retrieved?” columns are editable
- **Column Type (getColumnClass)**
Ensures that the last two columns are treated as Boolean values, allowing them to appear as checkboxes in the table.

```
static DefaultTableModel Table1 = new DefaultTableModel(null, Columns) {
    @Override
    public boolean isCellEditable(int row, int column) { //every column will be uneditable except for "found?" and "retrieved?"
        if (ci.getState()) { //if "ci" checkbox is checked
            return true;
        } else {
            return column >= 5;
        }
    }
    @Override
    public Class getColumnClass(int Index) { //"found?" and "retrieved?" will be boolean checkboxes
        Class c = String.class;
        if (Index >= 5) {
            return Boolean.class;
        }
        return c;
    }
    public boolean setCellSelectionEnabled () {
        return true;
    }
};
```

4. Add Record Window (Form Logic)

The `AddWindow();` method creates a separate form interface where users can input item details. It uses:

- `JTextField` for short inputs
- `JTextArea` for descriptions

```
public static void AddWindow() { //window for detail prompt
    JFrame f3 = new JFrame("New Item");
    JPanel p = new JPanel();

    Toolkit k = Toolkit.getDefaultToolkit();
    Image Icon = k.createImage("l&f.png");
    f3.setIconImage(Icon);

    JLabel l = new JLabel("Name of Requester:"); //labels for textboxes
    JLabel l2 = new JLabel("Item Name:");
    JLabel l3 = new JLabel("Item Description:");
    JLabel l4 = new JLabel("Current Date:");
    JLabel l5 = new JLabel("Last known location:");

    JTextField NameT = new JTextField(); //textfields for details
    JTextField ItemT = new JTextField();
    JTextArea DescT = new JTextArea();
    DescT.setLineWrap(true); //description textarea will wrap text if it reaches the edge
    DescT.setWrapStyleWord(true); //wrap by word instead of by char
    JTextField DateT = new JTextField();
    JTextField LocT = new JTextField(); // "T" indicates it is from a textfield/area
}
```

A **DocumentListener** is implemented to validate input fields in real time. The submit button is only enabled when all required fields are filled. Once submitted, the data is added as a new row in the table.

```
DocumentListener d = new DocumentListener() { //check if textfields are empty or not
    @Override
    public void changedUpdate(DocumentEvent e) {
        Empty();
    }
    @Override
    public void removeUpdate(DocumentEvent e) {
        Empty();
    }
    @Override
    public void insertUpdate(DocumentEvent e) {
        Empty();
    }

    public void Empty() {
        if (DateT.getText().isBlank() || ItemT.getText().isBlank() || DescT.getText().isBlank() || NameT.getText().isBlank()) {
            SUBMIT.setEnabled(false); //disable submit button if blank
        } else {
            SUBMIT.setEnabled(true); //enable if not
        }
    }
}
```

5. JSON Data Storage and File Handling

The system uses the **json-simple library** for saving and loading data.

- **Data Storage (JSON Arrays)**

```
static JSONArray Date, Item, Desc, Name, Loc, Found, Ret;
```

These arrays temporarily store table data before writing to a file.

```
static JSONArray Date = new JSONArray(); //Date Requested/Found
static JSONArray Item = new JSONArray(); //Item Name
static JSONArray Desc = new JSONArray(); //Description
static JSONArray Name = new JSONArray(); //Requester
static JSONArray Loc = new JSONArray(); //Last Known Location
static JSONArray Found = new JSONArray(); //Found?
static JSONArray Ret = new JSONArray(); //Retrieved?
```

Save Function (**SaveWindow();**)

- Opens a file chooser dialog

```
public static void SaveWindow() { //window for saving
    JFileChooser fc = new JFileChooser();
    fc.setAcceptAllFileFilterUsed(false);
    fc.setFileFilter(new FileNameExtensionFilter(".json, *.JSON", "json"));

    int Confirm = fc.showSaveDialog(f);
    if (Confirm == JFileChooser.APPROVE_OPTION) {
```

- Collects table data into JSON arrays

```
for (int j = 0; j < MainTable.getRowCount(); j++) {
    Date.add(MainTable.getValueAt(j, 0));
    Item.add(MainTable.getValueAt(j, 1));
    Desc.add(MainTable.getValueAt(j, 2));
    Name.add(MainTable.getValueAt(j, 3));
    Loc.add(MainTable.getValueAt(j, 4));
    Found.add(MainTable.getValueAt(j, 5));
    Ret.add(MainTable.getValueAt(j, 6));
}
```

- Converts data into a **JSONObject**

```
JSONObject o = new JSONObject(); //put everything into a JSONObject to be written to a file
o.put("Date Requested", Date);
o.put("Item Name", Item);
o.put("Item Description", Desc);
o.put("Requester", Name);
o.put("Last Known Location", Loc);
o.put("Found", Found);
o.put("Retrieved", Ret);
```

- Writes the data to a file using **FileWriter**
- Automatically ensures the **.json** file extension

```
if (IsJson) { //if it has ".json"
    try (FileWriter w = new FileWriter(String.valueOf(fc.getSelectedFile()))) {
        w.write(o.toString());
        w.flush();
    } catch (IOException ex) {
        ex.printStackTrace();
    }
} else { //if not
    try (FileWriter w = new FileWriter(fc.getSelectedFile() + ".json")) { //add ".json" to the end
        w.write(o.toString());
        w.flush();
    } catch (IOException ex) {
        ex.printStackTrace();
    }
}
```

Load Function (**LoadWindow();**)

- Opens a file chooser to select a JSON file

```
public static void LoadWindow() { //jframe for saving
    JFileChooser fc = new JFileChooser();
    fc.setAcceptAllFileFilterUsed(false);
    fc.setFileFilter(new FileNameExtensionFilter(".json, *.JSON", "json")); //same as SaveWindow()

    int Confirm = fc.showOpenDialog(f);
```

- Uses **JSONParser** to read and parse the file

```
JSONParser Jp = new JSONParser();
```

```
try (FileReader r = new FileReader(fc.getSelectedFile())) {
    JSONObject o = (JSONObject) Jp.parse(r);

    Date = (JSONArray)o.get("Date Requested"); //get the table values from the selected json file
    Item = (JSONArray)o.get("Item Name");
    Desc = (JSONArray)o.get("Item Description");
    Name = (JSONArray)o.get("Requester");
    Loc = (JSONArray)o.get("Last Known Location");
    Found = (JSONArray)o.get("Found");
    Ret = (JSONArray)o.get("Retrieved");
    Table1.setRowCount(0); //clear table first
}
```

- Extracts stored arrays and repopulates the table

```
for (int i = 0; i < Date.size(); i++) { //get all table values from json arrays and put them in the table
    Object NewRow[] = {Date.get(i), Item.get(i), Desc.get(i), Name.get(i), Loc.get(i), Found.get(i), Ret.get(i)};
    Table1.addRow(NewRow);
}
```

- Includes error handling for:
 - File not found
 - Invalid or corrupted JSON format

```
} catch (FileNotFoundException ex) {
    JOptionPane.showMessageDialog(f, "This file does not exist!", "Lost and Found", JOptionPane.ERROR_MESSAGE);
} catch (IOException ex) {
    ex.printStackTrace();
} catch (ParseException ex) {
    JOptionPane.showMessageDialog(f, "This file is invalid.", "Lost and Found", JOptionPane.ERROR_MESSAGE);
}
} else { //if not
    JOptionPane.showMessageDialog(f, "This file does not exist!", "Lost and Found", JOptionPane.ERROR_MESSAGE); //end
}
```

6. Search Implementation

The system implements dynamic search using:

- **TableRowSorter**
- **RowFilter.regexFilter()**

```

JTextField tf = new JTextField();
DocumentListener d = new DocumentListener() { //checks when the textfields are updated
    @Override
    public void changedUpdate(DocumentEvent e) {
        Search();
    }
    @Override
    public void removeUpdate(DocumentEvent e) {
        Search();
    }
    @Override
    public void insertUpdate(DocumentEvent e) {
        Search();
    }
}

public void Search() {
    TableRowSorter s = new TableRowSorter(Table1);
    String search = (tf.getText());
    s.setRowFilter((RowFilter.regexFilter("(?i)" + search))); //"(?i)" makes the regex filter case-insensitive
    MainTable.setRowSorter(s);
}

```

This allows real-time filtering of table data based on user input. The search is case-insensitive and updates automatically as the user types.

7. Event Handling System

The program relies on multiple listeners to handle user interaction:

- **ActionListener** – Handles button clicks and menu actions

```

JButton SUBMIT = new JButton("SUBMIT"); //submit
SUBMIT.addActionListener(e -> {
    Object NewRow[] = {(DateT.getText()), (ItemT.getText()), (DescT.getText()), (NameT.getText()), "", false, false};
    if (LocT.getText().isBlank()) { //in case Location is left blank, replace with "N/A"
        NewRow[4] = ("N/A");
    } else {
        NewRow[4] = LocT.getText();
    }
    f3.setVisible(false);

    Table1.addRow(NewRow);
});

```

- **DocumentListener** – Monitors changes in text fields (used for validation and search)

```

DocumentListener d = new DocumentListener() { //check if textfields are empty or not
    @Override
    public void changedUpdate(DocumentEvent e) {
        Empty();
    }
    @Override
    public void removeUpdate(DocumentEvent e) {
        Empty();
    }
    @Override
    public void insertUpdate(DocumentEvent e) {
        Empty();
    }
}

```

- **ListSelectionListener (ls)** – Tracks selected table rows and enables or disables the delete button accordingly

```

static ListSelectionListener ls = new ListSelectionListener() {
    @Override
    public void valueChanged(ListSelectionEvent e) {
        //check for which row is selected
        SelectR = MainTable.getSelectedRow();
        DELETEROW.setEnabled(true);

        if (SelectR < 0) { //disables button when index is -1 (default when nothing is selected)
            DELETEROW.setEnabled(false);
            DELETEROW.setText("DELETE ROW");
        } else {
            DELETEROW.setText("DELETE ROW: " + SelectR);
        }
    }
};

```

8. UI Layout and Design

The layout of the application is organized using:

- **GridLayout**
Used in the AddWindow(); method to align form components neatly
- **BorderLayout**
Used in the main frame:
 - Center → Table
 - Bottom → Toolbar

Additionally, a custom cell renderer is implemented to allow **text wrapping** inside table cells for better readability.


```

public static class LineWrapCellRenderer extends JTextArea implements TableCellRenderer { //lets text in cells wrap when hitting edge
    @Override
    public Component getTableCellRendererComponent (JTable table, Object value, boolean isSelected, boolean hasFocus, int row, int column) {
        this.setText((String)value);
        this.setWrapStyleWord(true);
        this.setLineWrap(true);
        return this;
    }
}

```

9. Toolbar Features

public static void ToolBar()

```

public static void ToolBar() {
    JToolBar t = new JToolBar();
    t.setFloatable(false);

    JLabel l = new JLabel ("Search Current Table: ");
    t.add(l);
    t.addSeparator();

    JTextField tf = new JTextField();
}

```

The toolbar provides quick access to essential functions:

- Search field for real-time filtering
- Delete button for removing selected rows

```

DELETEROW.addActionListener((ActionEvent e) -> {
    Table1.removeRow(MainTable.convertRowIndexToModel(SelectR)); //deletes a single selected row
});
DELETEROW.setEnabled(false);
t.add(DELETEROW);

```

- Dynamic updates based on user interaction

10. Menu System

public static void MenuBar()

The menu bar organizes system functions into:

- **File Menu** – New, Save, Load, Exit
- **Edit Menu** – Add row, Delete row, Toggle edit mode
- **About Menu** – Displays system and developer information

Keyboard shortcuts are implemented to improve efficiency.

11. Main Method

```
public static void main(String[] args)
```

This is the entry point of the application. It initializes all major components:

- Frame (main window)
- Table (data display)
- Menu bar (navigation)
- Toolbar (tools)

After initialization, the main window is displayed to the user.

```
public static void main(String[] args) {  
    Frame();  
    Table();  
    MenuBar();  
    ToolBar();  
  
    f.setVisible(true);  
}
```

12. Additional Technical Features

The system incorporates several supporting technical features that enhance usability and overall performance:

- **Custom Cell Renderer for Text Wrapping**
A custom cell renderer is implemented to allow long text (such as item descriptions) to automatically wrap within table cells. This improves readability and ensures that important information is fully visible without requiring horizontal scrolling.
- **Dynamic Enabling and Disabling of Buttons**
Buttons such as the “Delete Row” button are dynamically enabled or disabled based on user actions. For example, the delete button is only activated when a row is selected, helping prevent accidental operations and improving user interaction.
- **Keyboard Shortcut Integration**
The system includes keyboard shortcuts (e.g., Ctrl + S for saving, Ctrl + O for loading, Ctrl + N for new table) to provide faster navigation and improve efficiency for experienced users.

- **Regular Expression Validation for File Extensions**

A regular expression (regex) is used to verify whether the selected file name already contains the “.json” extension. If not, the system automatically appends it, ensuring correct file formatting and preventing saving errors.

E. System Requirements

To ensure smooth operation of the system, the following minimum requirements should be met:

Software Requirements

- **Java Development Kit (JDK) 8 or higher**

Required to compile and run the application, as it utilizes Java Swing components and standard libraries.

- **Java IDE (NetBeans recommended)**

An Integrated Development Environment is needed for running and managing the project. NetBeans is recommended for its compatibility with Swing-based applications.

- **json-simple library**

This external library is required for handling JSON-based data storage, including saving and loading records.

Hardware Requirements

- **Processor:** 1.0 GHz or higher

Ensures that the system can run the application without lag.

- **RAM:** Minimum 512 MB (1 GB recommended)

Provides sufficient memory for smooth GUI performance and data handling.

- **Storage:** At least 50 MB of free disk space

Required for the application files and saved JSON data.

- **Display:** Minimum resolution of 1024 × 768

Ensures proper layout and visibility of all interface components.

F. Summary

The **Lost and Found Management System** is a practical and user-friendly application that demonstrates key programming concepts such as graphical user interface development using Java Swing, event-driven programming, file handling through JSON, and basic data validation techniques.

Despite its relatively simple structure, the system effectively addresses real-world needs by providing an organized and efficient way to manage lost and found items. It allows users to easily record, update, search, and track item information within a centralized interface.

Its lightweight design, intuitive layout, and responsive features make it suitable for everyday use in school environments, particularly for managing item records in a fast, reliable, and organized manner.

Comparative Analysis

This section compares the developed **Lost and Found Management System** with existing applications to identify similarities, differences, and unique features.

A. Existing Applications

1. iLost

A cloud-based lost and found platform used by organizations such as airports and hotels.

Key Features:

- Online reporting of lost and found items
- Public search access for users
- Categorization and filtering of items
- Customer self-service portal

Comparison:

The developed system differs because it is **offline and desktop-based**, meaning it does not require internet access. While iLost provides broader accessibility and user interaction, the proposed system focuses on **internal use within a school**, making it simpler and more controlled. However, it lacks features like public search and online claiming.

2. ReclaimHub

A modern cloud-based system designed to manage and return lost property efficiently.

Key Features:

- Automatic matching of lost and found items
- Image upload for item identification

- Notification system for item recovery
- Reporting and analytics tools

Comparison:

Compared to ReclaimHub, the proposed system is **less complex but more accessible** in offline environments. ReclaimHub offers advanced automation (e.g., auto-matching and notifications), while the developed system relies on **manual searching and status tracking**, making it easier to use but less automated.

3. Spreadsheet-Based Systems (e.g., Microsoft Excel)

Commonly used in schools for recording lost items.

Key Features:

- Manual data entry
- Basic sorting and filtering
- Flexible but unstructured data storage

Comparison:

The proposed system improves significantly over spreadsheets by offering:

- Structured input fields
- Real-time search functionality
- Status tracking (Found / Retrieved)
- Input validation

Unlike spreadsheets, which are prone to human error, the system provides a **more organized and user-friendly interface**.

B. Innovative Features of the Proposed System

Despite being simple, the developed system includes useful innovations:

- **Offline Functionality**
Operates without internet, making it ideal for schools with limited connectivity.
- **Real-Time Search and Filtering**
Allows quick retrieval of records, improving efficiency.
- **Checkbox-Based Status Tracking**
Provides an easy way to monitor whether items are found or claimed.
- **Editable Control System**
Restricts or allows editing to maintain data accuracy.
- **Lightweight JSON Storage**
Eliminates the need for complex databases while maintaining portability.

C. Overall Comparison

Compared to existing systems, the developed application emphasizes **simplicity, accessibility, and usability**. While modern platforms like iLost and ReclaimHub offer advanced features such as automation, image recognition, and online access, they require internet connectivity and are more complex.

In contrast, the proposed system provides a **practical and efficient solution for small-scale environments**, particularly schools, by focusing on ease of use and offline capability. It bridges the gap between **manual methods (e.g., spreadsheets)** and **advanced cloud-based systems**, offering a balanced and appropriate solution for its target users

Testing and Debugging

TEST CASE	PRE-CONDITION(S)	PROCEDURE(S)	SAMPLE TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS
Add New Row (Validation)	- Must be on the main screen	1. Click "Edit" -> "Add New Row" 2. Leave required fields blank 3. Observe SUBMIT button	Name: (blank) Item: (blank)	The "SUBMIT" button remains disabled until all required fields are filled.	The SUBMIT button remained greyed out and unclickable while fields were empty.	Passed
Add New Row (Success)	- Must be on the "New Item" window	1. Fill in Requester, Item, Description, and Date 2. Leave Location blank 3. Click "SUBMIT"	Name: "John Doe" Item: "Keys" Location: (blank)	A new row is added to the table. The blank Location is automatically set to "N/A".	Row was successfully appended to the bottom of the table, with Location showing "N/A".	Passed

Delete Row	-	A row must be selected in the main table	1. Click a row to select it 2. Click the "DELETE ROW" button (or press Del)	Row Index: 1	The selected row is removed from the table.	The selected row disappeared from the view immediately upon clicking the button.	Passed ▾
Make Table Editable	-	Must be on the main screen with existing table data	1. Click "Edit" -> "Make Table Editable" 2. Double-click any text cell to edit	N/A	The table cells become editable, allowing the user to type directly into them.	Users were able to double-click and freely modify the text in the previously locked columns.	Passed ▾
Save to JSON	-	Must be on the main screen	1. Click "File" -> "Save to JSON" 2. Enter a filename 3. Click Save	lost_items_2026	A JSON file is generated containing the table data. The .json extension is appended automatically.	The file lost_items_2026.json was successfully created in the target directory with accurate array data.	Passed ▾
Load from JSON	-	Must be on the main screen	1. Click "File" -> "Load from JSON" 2. Click "Yes" on the confirmation prompt 3. Select a valid file	lost_items_2026.json	The table is cleared and populated with the data from the imported JSON file.	The application successfully cleared the current table and correctly populated the rows from the file.	Passed ▾
Search Table	-	Table must contain multiple rows of data	1. Click the "Search Current Table" text field 2. Type a keyword	"Keys"	The table filters case-insensitively to display only rows containing the searched	The table dynamically updated to hide non-matching rows and only displayed	Passed ▾

				keyword.	items containing "keys".	
New Table	- Must be on the main screen	1. Click "File" -> "New Table" 2. Click "Yes" on the confirmation prompt	N/A	The table is completely cleared of all rows.	All table data was wiped clean returning the grid to a default empty state.	Passed

Reflection and Enhancements

RAMON BENEDICT R. ELIZAN

I had quite the experience with creating this program. As the main programmer, I had to do most of the coding work, implementation of features, and the list of features the program would have in the first place. At first, our scope was just far too large, and I had to trim the more unnecessary features in order to get the program completed within the allotted timeframe. Additionally, my early implementation of certain functions were awful, since I had not yet done enough research on Swing components. In short, most of the old implementations were far from user-friendly.

An example that comes to mind as I am writing this is the load function. Before, the user had to type the JSON file's file name. It could not access different directories, hinged on the user's ability to remember file names, and was case-sensitive. After discovering JFileChooser, the loading is now similar to other modern programs, in which the user can simply click on the file they wish to load in a window.

Despite all the hangups, I did have fun putting the code together to create a functional program. There is just something I truly love about the process of finding creative solutions and implementations, even with all of the problems that come with it. There is just nothing that compares to the rush that comes with finally fixing that one issue, or even implementing something as basic as deleting a row on the table.

As for updates to the program, I would suggest a popup menu for the main window, since that is missing as of currently. Though, I struggled to implement it myself, since the table is a separate element from the frame, and would need its own trigger for the popup menu. I just did not want to deal with the possible bugs that would come with that. Next, I would suggest a proper main menu, something like the Microsoft Office programs have, since the program just

boots into an empty table right now. Finally, I would suggest the undo/redo and copy/paste functions that normally come with other programs.

JOHANNA EZRA IRANON

In our project, I took on multiple roles including handling test cases, quality assurance (QA), creating the PowerPoint presentation, and serving as the project manager. As the project manager, I helped organize tasks, ensured deadlines were met, and coordinated with my groupmates. For QA and test cases, I checked the system for errors and made sure all features worked correctly. I also prepared the presentation to clearly explain our system.

Overall, building this project was a great experience. I learned a lot about teamwork, time management, and applying programming ideas in a real-world setting. Sure, there were challenges like troubleshooting errors or figuring out tricky parts of the system but overcoming them made the process rewarding.

For the future, I think our system could be even better by using a proper database instead of JSON to handle data more efficiently, adding user login for extra security, and giving the interface a more modern look. Features like search filters and notifications could also make the system easier and faster to use.

ART XENOS D. GUADALQUIVER

In this project, my main role was to handle the documentation of the system. I was responsible for organizing the content, writing the system documentation, and ensuring that all parts of the project were clearly explained and properly structured. This included creating sections such as the introduction, user manual, feature list, and code documentation.

At first, I thought documentation would be a simple task, but I realized that it required a deep understanding of how the system actually works. I had to carefully analyze the program, understand the flow of each feature, and translate technical concepts into clear and understandable explanations. One challenge I encountered was making sure that the documentation was detailed enough to be informative, but not too complicated for readers to understand.

I also had to make sure that the format and style of our documentation were consistent and aligned with academic standards. This required attention to detail, especially in organizing sections, improving clarity, and refining the wording of each part. Additionally, I worked closely with my groupmates to ensure that the documentation accurately reflected the features and behavior of the system.

Overall, this experience helped me improve my technical writing skills and taught me the importance of clear communication in software development. I realized that even a well-developed system can be difficult to use or understand without proper documentation.

For future improvements, I suggest enhancing the documentation further by including visual aids such as flowcharts or system diagrams to better illustrate how the system works. Adding screenshots of the interface could also help users understand the steps more easily. Additionally, providing a more detailed explanation of the system architecture and design decisions would make the documentation more comprehensive and professional.